

Московский авиационный институт  
(Национальный исследовательский университет)  
Институт №8 «Компьютерные науки и прикладная математика»

**Лабораторная работа №4**  
по курсу «Теория сложности алгоритмов»  
Вариант №2

Выполнила студентка: Бондарева Е. Е.

Группа: М8О-109СВ-25

Преподаватель: Рассказова В. А.

Дата: \_\_\_\_14.04.2026\_\_\_\_

Оценка: \_\_\_\_\_

Москва, 2026

## Задание

Для заданного языка

1. построить описание МТ с полиномиальной временной сложностью, решающей его;
2. построить оценку сложности данной МТ;
3. реализовать данную МТ в виде программы;
4. провести тестовые исследования, демонстрирующие совпадение фактической временной сложности с теоретической.

$\text{TRIANGLE} = \{ \langle G \rangle : G \text{ -- неориентированный граф, содержащий 3-клику} \}$

## Теоретическая часть

Распознаваемые языки -- существует МТ, которая их распознает (может заиклиться).

Разрешимые языки -- существует МТ, которая их решает (останавливается на любом входе с ответом да или нет). Иными словами для разрешимых языков возможна реализация алгоритма полного перебора.

Пример неразрешимого языка -- 10-ая проблема Гилберта о решении алгебраических уравнений в целых числах (если на входе полином с целочисленными корнями, то МТ может остановиться и ответить “да” если эти корни найдены, или же может выполнять бесконечные вычисления не ответив тем самым ни да, ни нет).

**Теорема 4.1.** Язык  $A_{TM}$  неразрешим по Тьюрингу (МТ допускает строку, если останавливается, но если не останавливается и заикливается -- то нельзя с уверенностью утверждать, будет ли строка допущена или отвергнута, то есть имеет место свойство частичной разрешимости).

Доказательство. Предположим, что  $A_{TM}$  разрешим и существует МТ решатель  $R$ . Построим на основе  $R$  МТ  $D$ , которая проверяет, допускает ли свое собственное описание:  $D = \text{на входе } \langle M \rangle$ :

1. Запустить  $R$  на входе  $\langle M, M \rangle$
2. Если  $R$  допускает, то отвергнуть, иначе допустить.

Тогда по построению вход  $\langle D \rangle$  будет допущен  $D$  в том и только в том случае, когда  $D$  не допускает. Получаем противоречие, следовательно, предположение о существовании  $R$  неверно.

**Теорема 4.2.** Существуют нераспознаваемые языки.

Доказательство. Для доказательства сопоставим множество всех возможных МТ и множество всех языков (поскольку если все языки распознаваемы, значит для всех существует МТ). Поскольку множество всех строк  $\Sigma^*$  счетно, то множество всех языков  $\wp(\Sigma^*)$  имеет мощность континуум (несчетно). При этом каждая МТ может быть представлена в виде строки (ее описание). Таким образом необходимо установить взаимно однозначное соответствие между счетным множеством МТ и несчетным множеством всех языков, что невозможно.

**Теорема 4.3. (Поста).** Язык  $A$  разрешим тогда и только тогда, когда  $A$  и  $\bar{A}$  распознаваемы. Доказательство. Пусть  $A$  разрешим (значит распознаваем) и  $M$  -- решатель  $A$ . Тогда  $M'$ , которая симулирует  $M$  на своем входе и отвечает противоположное, будет решать  $\bar{A}$  (значит распознаваем).

Пусть теперь  $A$  и  $\bar{A}$  распознаваемы и  $M'$  и  $M''$  их распознают. Покажем, что в этом случае  $A$  разрешим. Построим решатель  $M$  для  $A$ :  $M = \text{на входе } \omega$ :

1. Запустить  $M'$  и  $M''$  параллельно
2. Если  $M'$  допускает, то допустить (строка принадлежит  $A$ ).

3. Если  $M''$  допускает, то отвергнуть (строка принадлежит  $\bar{A}$ ).

Поскольку  $\omega \in A \cup \bar{A}$ , то  $M$  остановится за конечное число шагов (поскольку остановится либо  $M'$ , либо  $M''$ ). Значит,  $M$  – решатель

**Опр.4.1** Пусть  $M$  -- детерминированная МТ решатель. Тогда временем работы или временной сложностью (по Колмогорову) называется функция  $f: \mathbb{N} \rightarrow \mathbb{N}$ , где  $f(n)$  -- максимальное количество тактов МТ  $M$ , которые она совершит на входе  $\omega \in \Sigma^*$ ,  $|\omega|=n$ .

**Опр.4.2** Пусть  $t: \mathbb{N} \rightarrow [0, \infty)$ . Тогда

$$\text{TIME}(t(n)) = \left\{ A \subset \Sigma^* : \text{существует ДМТ, которая решает } A \text{ за время } O(t(n)) \right\}$$

**Опр.4.3** Классом  $P$  называют класс языков, разрешимых за полиномиальное время на детерминированной машине Тьюринга:

$$P = \bigcup_{k=1}^{\infty} \text{TIME}(n^k)$$

Через  $\text{PATH}$  обозначим задачу обнаружения пути из одной вершины в другую в ориентированном графе  $G$ :

$$\text{PATH} = \{ \langle G, s, t \rangle : G \text{ -- граф, в котором есть путь из } s \text{ в } t \}$$

**Теорема 4.4.** Язык  $\text{PATH} \in P$ .

Доказательство. Язык  $\text{PATH}$  разрешим, поскольку можно перебрать все пути в графе (их в экстремальном варианте  $n!$ ) и проверить, есть ли среди них путь из  $s$  в  $t$ . Однако кроме полного перебора для данного языка существуют детерминированные полиномиальные МТ (поиск в глубину или в ширину). Следовательно, данный язык лежит в классе  $P$ :

1. пометить вершину  $s$ ,

2. пометить все вершины, достижимые за один шаг из уже помеченных вершин,
3. повторять шаг 2 до тех пор, пока не перестанут появляться вновь помеченные вершины,
4. если вершина  $t$  помечена, то допустить.

Шаги 1 и 4 имеют сложность  $O(n)$ . В шаге 2 для каждой помеченной вершины нужно проверить все остальные на предмет достижимости (смежные вершины). Тогда сложность шага 2  $O(n^2)$ . При этом шаг 2 повторяется в рамках шага 3 не более  $O(n)$  раз. Тогда итоговая сложность  $O(n) + O(n^2) \cdot O(n) + O(n) = O(n^3)$  -- полином от размера входа.

## Код программы

"""

Лабораторная работа №4  
Исследование временной сложности

Язык:  $\text{TRIANGLE} = \{ \langle G \rangle : G \text{ — неориентированный граф, содержащий 3-клику} \}$

Описание МТ:

Вход: матрица смежности графа  $G$  размера  $n \times n$  на ленте.

Алгоритм (детерминированная МТ):

1. Перебрать все тройки вершин  $(i, j, k)$ , где  $0 \leq i < j < k < n$ .
2. Для каждой тройки проверить наличие рёбер  $(i, j)$ ,  $(j, k)$ ,  $(i, k)$ .
3. Если все три ребра есть — принять (АССЕРТ).
4. Если ни одна тройка не образует треугольник — отвергнуть (РЕЈЕСТ).

Число троек:  $C(n, 3) = n(n-1)(n-2)/6$ .

Каждая проверка —  $O(1)$  обращений к матрице смежности.

Итого временная сложность:  $O(n^3)$ .

Теоретическая оценка шагов МТ:

$T(n) \approx c \cdot n^3$ , где  $c$  — константа ( $\approx 1/6$  для полного перебора).

"""

```
import time
```

```

import matplotlib.pyplot as plt

# --- Реализация MT ---

def has_triangle(adj, n):
    """
    Моделирует работу MT: перебирает все тройки вершин,
    проверяет наличие треугольника. Возвращает (результат,
    число шагов).
    """
    steps = 0
    for i in range(n):
        for j in range(i + 1, n):
            for k in range(j + 1, n):
                steps += 1
                if adj[i][j] and adj[j][k] and adj[i][k]:
                    return True, steps
    return False, steps

# --- Генерация графов ---

def graph_without_triangle(n):
    """Двудольный граф (гарантированно без треугольников)."""
    adj = [[False] * n for _ in range(n)]
    half = n // 2
    for i in range(half):
        for j in range(half, n):
            adj[i][j] = adj[j][i] = True
    return adj

def complete_graph(n):
    """Полный граф (треугольник найдётся на первом шаге)."""
    return [[i != j for j in range(n)] for i in range(n)]

# --- Теоретическая оценка ---

def theoretical_steps(n):
    """C(n,3) — worst-case число шагов."""
    return n * (n - 1) * (n - 2) // 6

def visualization(sizes, worst_steps, worst_theory,
                  worst_times):
    # --- Визуализация ---
    fig, axes = plt.subplots(1, 3, figsize=(16, 5))

    # График 1: факт vs теория (worst-case)

```

```

ax = axes[0]
ax.plot(sizes, worst_steps, 'ro-', label='Факт. шаги')
ax.plot(sizes, worst_theory, 'b--s', label='C(n,3) =
n(n-1)(n-2)/6')
ax.set_xlabel('n (число вершин)')
ax.set_ylabel('Число шагов')
ax.set_title('Worst-case: факт vs теория')
ax.legend()
ax.grid(True, alpha=0.3)

# График 2: шаги/n³ → константа
ax = axes[1]
ratios = [s / (n ** 3) for s, n in zip(worst_steps, sizes)]
ax.plot(sizes, ratios, 'go-', linewidth=2)
ax.axhline(y=1 / 6, color='r', linestyle='--', label='1/6 ≈
0.1667')
ax.set_xlabel('n (число вершин)')
ax.set_ylabel('Шаги / n³')
ax.set_title('Шаги/n³ → const (подтверждение O(n³))')
ax.legend()
ax.grid(True, alpha=0.3)

# График 3: время выполнения
ax = axes[2]
ax.plot(sizes, worst_times, 'mo-', label='Время (мс)')
# Нормируем теоретическую кривую по последней точке
c = worst_times[-1] / (sizes[-1] ** 3)
theory_time = [c * n ** 3 for n in sizes]
ax.plot(sizes, theory_time, 'b--s', label=f'c·n³
(c={c:.2e})')
ax.set_xlabel('n (число вершин)')
ax.set_ylabel('Время (мс)')
ax.set_title('Время выполнения vs c·n³')
ax.legend()
ax.grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig('lab4_triangle_plot.png', dpi=150)
print(f"\nГрафик сохранён: lab4_triangle_plot.png")
plt.show()

# --- Тесты и визуализация ---

def run_tests():
    sizes = [10, 20, 40, 60, 80, 100, 150, 200]

    # --- Тест 1: worst-case (без треугольников) ---
    worst_steps = []
    worst_theory = []
    worst_times = []

```

```

    print("=" * 72)
    print("ТЕСТ 1: Worst-case (двудольный граф, без
треугольников)")
    print("-" * 72)
    print(f"{ 'n':>5} | { 'Факт.шаги':>11} | { 'C(n,3)':>11} |
{ 'Факт/C(n,3)':>11} | { 'Время,мс':>9}")
    print("-" * 72)

    for n in sizes:
        adj = graph_without_triangle(n)
        t0 = time.perf_counter()
        result, steps = has_triangle(adj, n)
        dt = (time.perf_counter() - t0) * 1000
        th = theoretical_steps(n)
        worst_steps.append(steps)
        worst_theory.append(th)
        worst_times.append(dt)
        print(f"{n:>5} | {steps:>11} | {th:>11} |
{steps/th:>11.4f} | {dt:>9.2f}")

    # --- Тест 2: best-case (полный граф) ---
    best_steps = []

    print()
    print("=" * 72)
    print("ТЕСТ 2: Best-case (полный граф, треугольник сразу)")
    print("-" * 72)
    print(f"{ 'n':>5} | { 'Факт.шаги':>11} | { 'C(n,3)':>11} |
{ 'Результат':>10}")
    print("-" * 72)

    for n in sizes:
        adj = complete_graph(n)
        result, steps = has_triangle(adj, n)
        best_steps.append(steps)
        print(f"{n:>5} | {steps:>11} |
{theoretical_steps(n):>11} | { 'АССЕПТ':>10}")

    # --- Тест 3: проверка кубического роста ---
    print()
    print("=" * 72)
    print("ТЕСТ 3: Проверка кубического роста (worst-case)")
    print("-" * 72)
    print(f"{ 'n':>5} | { 'Шаги':>11} | { 'Шаги/n³':>11} | { 'Рост
шагов':>11} | { '(n/n_prev)³':>11}")
    print("-" * 72)

    for idx, n in enumerate(sizes):
        s = worst_steps[idx]
        r = s / (n ** 3)

```



```

        if idx > 0:
            growth = s / worst_steps[idx - 1]
            expected = (n / sizes[idx - 1]) ** 3
            print(f"{n:>5} | {s:>11} | {r:>11.6f} |
{growth:>11.2f} | {expected:>11.2f}")
        else:
            print(f"{n:>5} | {s:>11} | {r:>11.6f} | {'-':>11} |
{'-':>11}")

# --- Тест 4: конкретные примеры ---
print()
print("=" * 72)
print("ТЕСТ 4: Конкретные примеры")
print("-" * 72)

cases = [
    (
        "Треугольник {0,1,2}",
        3,
        [
            (0, 1),
            (1, 2),
            (0, 2)
        ]
    ),
    (
        "Путь 0-1-2",
        3,
        [
            (0, 1),
            (1, 2)
        ]
    ),
    (
        "Пустой граф (4 верш.)",
        4,
        [
        ]
    ),
    (
        "5 верш., Δ на {2,3,4}",
        5,
        [
            (0, 1),
            (2, 3),
            (3, 4),
            (2, 4)
        ]
    ),
    (

```

```

        "Тестовый пример",
        5,
        [
            (0, 4),
            (0, 2),
            (0, 3),
            (1, 3)
        ]
    )
]
for name, n, edges in cases:
    adj = [[False]*n for _ in range(n)]
    for u, v in edges:
        adj[u][v] = adj[v][u] = True
    res, s = has_triangle(adj, n)
    print(f"    {name:<25} → {'ACCEPT' if res else 'REJECT'},
шагов: {s}")

# --- Вывод ---
print()
print("=" * 72)
print("Вывод:")
print("    Факт. число шагов в worst-case = C(n,3) = n(n-1)(n-2)/6.")
print("    Отношение шагов/n³ → 1/6 ≈ 0.1667 — подтверждает O(n³).")
print("    Рост при удвоении n ≈ 8×, что соответствует кубической сложности.")
print("=" * 72)

visualization(sizes, worst_steps, worst_theory, worst_times)

if __name__ == "__main__":
    run_tests()

```

## Тесты

ТЕСТ 1: Worst-case (двудольный граф, без треугольников)

| n   | Факт.шаги | C(n,3)  | Факт/C(n,3) | Время,мс |
|-----|-----------|---------|-------------|----------|
| 10  | 120       | 120     | 1.0000      | 0.01     |
| 20  | 1140      | 1140    | 1.0000      | 0.09     |
| 40  | 9880      | 9880    | 1.0000      | 0.68     |
| 60  | 34220     | 34220   | 1.0000      | 2.66     |
| 80  | 82160     | 82160   | 1.0000      | 4.57     |
| 100 | 161700    | 161700  | 1.0000      | 11.03    |
| 150 | 551300    | 551300  | 1.0000      | 22.47    |
| 200 | 1313400   | 1313400 | 1.0000      | 53.17    |

## ТЕСТ 2: Best-case (полный граф, треугольник сразу)

| n   | Факт.шаги | C(n, 3) | Результат |
|-----|-----------|---------|-----------|
| 10  | 1         | 120     | АССЕРТ    |
| 20  | 1         | 1140    | АССЕРТ    |
| 40  | 1         | 9880    | АССЕРТ    |
| 60  | 1         | 34220   | АССЕРТ    |
| 80  | 1         | 82160   | АССЕРТ    |
| 100 | 1         | 161700  | АССЕРТ    |
| 150 | 1         | 551300  | АССЕРТ    |
| 200 | 1         | 1313400 | АССЕРТ    |

## ТЕСТ 3: Проверка кубического роста (worst-case)

| n   | Шаги    | Шаги/n <sup>3</sup> | Рост шагов | (n/n_prev) <sup>3</sup> |
|-----|---------|---------------------|------------|-------------------------|
| 10  | 120     | 0.120000            | —          | —                       |
| 20  | 1140    | 0.142500            | 9.50       | 8.00                    |
| 40  | 9880    | 0.154375            | 8.67       | 8.00                    |
| 60  | 34220   | 0.158426            | 3.46       | 3.38                    |
| 80  | 82160   | 0.160469            | 2.40       | 2.37                    |
| 100 | 161700  | 0.161700            | 1.97       | 1.95                    |
| 150 | 551300  | 0.163348            | 3.41       | 3.38                    |
| 200 | 1313400 | 0.164175            | 2.38       | 2.37                    |

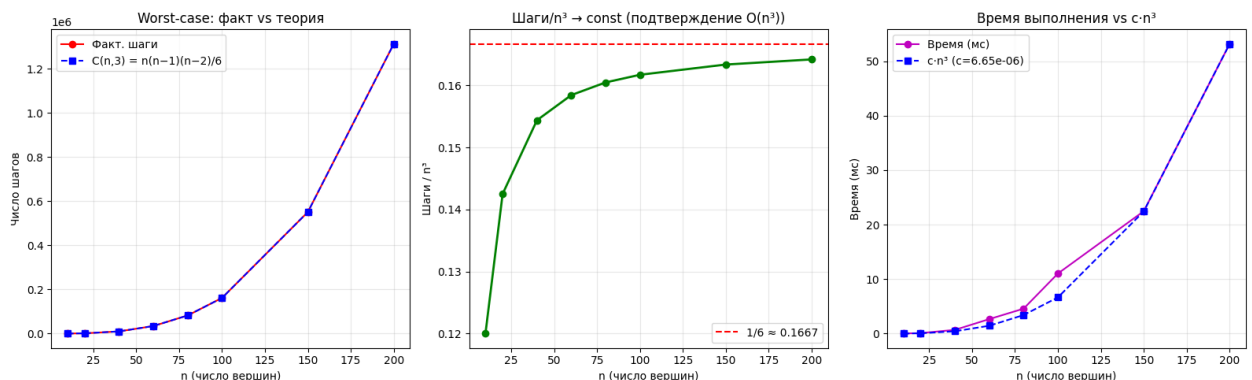
## ТЕСТ 4: Конкретные примеры

Треугольник {0,1,2} → АССЕРТ, шагов: 1  
 Путь 0-1-2 → РЕЖЕСТ, шагов: 1  
 Пустой граф (4 верш.) → РЕЖЕСТ, шагов: 4  
 5 верш., Δ на {2,3,4} → АССЕРТ, шагов: 10  
 Тестовый пример → РЕЖЕСТ, шагов: 10

## ВЫВОД:

Факт. число шагов в worst-case =  $C(n, 3) = n(n-1)(n-2)/6$ .  
 Отношение шагов/n<sup>3</sup> →  $1/6 \approx 0.1667$  — подтверждает  $O(n^3)$ .  
 Рост при удвоении  $n \approx 8\times$ , что соответствует кубической сложности.

График сохранён: lab4\_triangle\_plot.png



## **Вывод**

В результате выполнения лабораторной работы для заданного языка было построено описание МТ с полиномиальной временной сложностью, решающей его; была построена оценка сложности данной МТ; реализована данная МТ в виде программы и были проведены тестовые исследования, демонстрирующие совпадение фактической временной сложности с теоретической при условии, что  $\text{TRIANGLE} = \{\langle G \rangle : G \text{ – неориентированный граф, содержащий 3-клику}\}$ .